Verified Optimizations to Flock's RS Zerocheck Prover

measurements on Apple M1 Max, single-threaded, 2^{18} BLAKE3 compressions

branch `snark-fast-improvements`

August 2026

Abstract

Seven changes to the RS (additive-NTT) zerocheck path, each kept only after a paired, order-alternating A/B measurement with a decisive sign test. Together they reduce zerocheck round 1 from 1477 ms to 1205 ms (−18.4%), round 2 from 433 to 312 ms (−28%), the rounds-3+ tail from 455 to 307 ms (−33%), and whole-proof time by ≈ 11% single-threaded. Three are ideas ported from an independently optimized fork of this prover (the "challenge" tree), re-derived and re-implemented rather than copied; three originated here. The recurring themes: reductions in fields of characteristic two are $\mathbb{F}_2$-linear and can be deferred almost arbitrarily; work that a circuit fixes structurally can be skipped with a one-word guard; and on Apple silicon the boundary between the vector and general register files costs more than most instruction counts.

1 Setting and notation

Fields. $\mathbb{F}_{2^8} = \mathbb{F}_2[x]/(x^8 + x^4 + x^3 + x + 1)$ and the GHASH field $\mathbb{F}_{2^{128}} = \mathbb{F}_2[X]/(X^{128} + X^7 + X^2 + X + 1)$, with the ring embedding $\varphi : \mathbb{F}_{2^8} \hookrightarrow \mathbb{F}_{2^{128}}$. Write $\alpha := \varphi(x)$ and $\gamma := X$.

The witness is $z \in \mathbb{F}_2^{2^m}$; at the benchmark shape $m = 32$ (2^{18} BLAKE3 compressions, $k_{\log} = 14$ bits per block). After the univariate skip, round 1 of the zerocheck views the m index bits as

$$[\ \underbrace{\lambda}_{6\ \text{(univariate)}}\ |\ \underbrace{s}_{3\ \text{(small)}}\ |\ \underbrace{b}_{4\ \text{(medium)}}\ |\ \underbrace{t}_{m-13\ \text{(rest)}}\ ],$$

and must emit, for each of the 64 points λ of the skip domain, the quadratic message $P^{AB}(\lambda)$ and the linear message $P^{C}(\lambda)$, plus a pair of *banks* (the ring-switch helper $\hat{s}_{v,C}$) that keep the small parity dimension unfolded.

The protocol pins the small and medium challenges to *friendly* values: $r_{6+i} = \varphi(v_i)$ with $v = (\texttt{0xF7}, \texttt{0x53}, \texttt{0xB5})$, and $\beta_i = \gamma^{2^{i-1}}/(1 + \gamma^{2^{i-1}})$. These make the eq weights geometric:

$$\prod_{i=0}^{2} \mathrm{eq}(r_{6+i}, K_i) \;=\; C_s\, \alpha^K, \qquad \prod_{i=1}^{4} \mathrm{eq}(\beta_i, b_i) \;=\; \gamma^b / D, \tag{1}$$

with $K = \sum K_i 2^i$, $C_s = \prod_i (1 + r_{6+i}) = \varphi(\texttt{0x1C})$, and $D = \prod_i (1 + \gamma^{2^{i-1}})$. The baseline kernels exploit (1) through lookup tables; several optimizations below exploit it in the opposite direction, replacing tables by folds at the actual challenge values.

Measurement protocol. All numbers are single-threaded at the shape above. Each verdict is a paired A/B: both arms built once, one discarded warm-up each, then eight pairs with the *order alternating within each pair* (a fixed order is biased by thermal drift by roughly the size of the effects measured here). Phase times are the minimum over the ∼5 zerocheck invocations within one run. A change is kept only on a decisive sign test; two of the six wins were 8/8 with disjoint ranges. Measure only on AC power with the battery above ∼ 60%: low battery caps frequencies, and fast-charging a nearly empty battery is worse.

2 Deferred reduction in vector registers

For $u \in \mathbb{F}_2^{256}$ (an unreduced carry-less product) let $R(u) \in \mathbb{F}_{2^{128}}$ be reduction mod $X^{128} + X^7 + X^2 + X + 1$. R is $\mathbb{F}_2$-linear, so for any products $u_i = a_i \odot b_i$ (unreduced),

$$\textstyle\sum_i R(u_i) \;=\; R(\sum_i u_i)\,,$$

and a sum of n field products needs one reduction, not n. The baseline already used this identity, but held the 256-bit accumulator as a four-word struct in *general* registers: each product paid six lane extracts to leave the vector file, and each accumulate was four scalar XORs.

The fix is representational, not algorithmic: keep the accumulator as two 128-bit vector registers. The unreduced product uses Karatsuba,

$$(a_l + a_h X^{64})(b_l + b_h X^{64}) : \quad ll = a_l b_l, \quad hh = a_h b_h, \quad cross = (a_l{+}a_h)(b_l{+}b_h) + ll + hh,$$

three `PMULL`s instead of the schoolbook four; accumulation is two vector XORs; the four extracts are paid once per worker chunk on the way out, and the existing scalar reducer is reused unchanged.

Measured: round-2 phase 433.1 → 375.2 *ms (*−13.4%*).*

3 Structurally fixed rows of the b operand

Round 1's dominant kernel transforms the operands a, b through a table T implementing the inverse-NTT composition; T is $\mathbb{F}_2$-linear, so $T(0) = 0$. A census of the packed BLAKE3 witness (256 word positions per block × 256 blocks × 3 independent witnesses) shows the circuit pins 38 of 256 eight-byte K-rows of b regardless of the inputs, taking only three values:

$$\underbrace{\texttt{0xff}^8}_{\text{22 positions (const-one wires)}}\,, \qquad \underbrace{0^8}_{\text{15 positions (structural zeros)}}\,, \qquad \text{one mixed word.}$$

For a zero row the entire row's work vanishes: $db = T(0) = 0$, hence $y = da \odot db = 0$ contributes nothing to any accumulator. The kernel gains a single guard:

1: **if** $\text{load}_{64}(b\_row) = 0$ **then return** ▷ skips all 64 table loads and 4 $\mathbb{F}_{2^8}$ multiplies

No census data ships and no positions are tracked: the guard is exact for any witness, and a disagreeing witness falls through to the generic path. (The all-ones rows were also tried: constant-folding them saves only the b half of a row's work, and halving a row's loads halves its memory-level parallelism, returning 6 ms where 27 were predicted. Reverted; whole-row elimination is what pays.)

Measured: round 1 −42.4 *ms (*−2.9%*),* 8/8*.*

4 Instruction-level parallelism in the gather drain

The drain accumulates, per output lane, three XOR chains of depth $n_{\text{med}} = 16$ (one per bank), each link a table gather feeding an XOR. The gathers are independent but the chains are serial, so one lane exposes three dependency chains to the out-of-order engine. Processing two lanes per iteration doubles that to six with no change in total work:

```
for ℓ = 0, 2, 4, ..., 62 do
    u0, u1, u2 ← 0;   v0, v1, v2 ← 0          ▷ banks for lanes ℓ and ℓ+1
    for b = 0 ... n_med − 1 do
        u_j ⊕= convert[b][·];   v_j ⊕= convert[b][·]      ▷ 6 independent chains in flight
    end for
end for
```

This targets latency, not throughput: an attempt to shrink the gather *table* (64 → 8 KiB, doubling gather count) had cost 3.6% — the table already fit M1's 128 KiB L1D, establishing that the drain is bound by gather count and chain depth, not footprint.

Measured: round 1 −63.1 *ms (*−4.3%*),* 8/8*. Combined with Section 3:* 1477.3 → 1403.1 *ms (*−5.0%*).*

5 Unreduced accumulation with multiplicative weight splitting

The prep kernel computes, per K-row ($K \in \{0, \dots, 7\}$) and 16-lane block, $y_K = a_K \odot b_K \in \mathbb{F}_{2^8}$ and accumulates $\sum_K x^K y_K$ in 16-bit lanes, reducing once at the end. The baseline computed each y_K *reduced*: two `PMULL`s for the raw product plus a reduction costing four more, then a widening shift by K — six `PMULL`s per row-block for a reduction that the final fold makes redundant.

Let u_K be the *unreduced* 15-bit product. Then $\sum_K x^K u_K \equiv \sum_K x^K y_K \pmod{p_8}$, but $x^K u_K$ overflows 16 bits for $K \geq 2$. Split the weight so every factor lands where it is free:

$$x^K = \underbrace{(x^4)^{[K \geq 4]}}_{\text{choice of gather table}} \cdot \underbrace{(x^2)^{[\lfloor K/2 \rfloor \text{ odd}]}}_{\text{byte-wise doubling of the reduced operand}} \cdot \underbrace{x^{K \bmod 2}}_{\text{in-lane shift}}.$$

The x^4 factor costs nothing per element: a second table image with every byte pre-multiplied by x^4 scales each gathered XOR-sum by x^4, by linearity of T. The x^2 factor is a six-instruction shift-and-conditional-XOR on the already-reduced 8-bit operand (deg still ≤ 7). The residual shift is one vector instruction and raises term degree to at most 15.

Algorithm 1 Row-block accumulate, before and after

```
before: acc ⊕= shift_K(reduce(pmull(da, db)))                    ▷ 6 PMULL
after:  da ← [K ≥ 4] ? gathered from x^4 table : da;   da ← [(K/2) mod 2] ? x^2·da : da
        acc ⊕= shift_{K mod 2}(pmull(da, db))                    ▷ 2 PMULL
```

Correctness needs the final reducer to be exact to degree 15, one beyond its documented "≤ 14"; both the scalar and vector reducers were verified exhaustively over all 2^{16} inputs (tests now permanent). Gather traffic is unchanged apart from the second 16 KiB table image.

Measured: round 1 1401.4 → 1273.4 *ms (*−9.1%*),* 8/8*, every pair in* [−8.7%, −10.1%]*. The reference implementation's attribution predicted* 100–140 *ms; measured* 128*.*

6 The C-side message as a fold of the lincheck stripe

In the fast hash setups $C = I$, so $\hat{c} = \hat{z}$ and everything round 1 emits on the C side is *linear* in the witness. Concretely the two banks are, for parity $p \in \{0, 1\}$ and lane λ,

$$\mathrm{bank}_p[\lambda] \;=\; \sum_{K \equiv p \ (2)} \alpha^K \sum_b \frac{\gamma^b}{D} \sum_t \mathrm{eq}(r_{\geq 13}, t)\; z[\lambda, K, b, t], \tag{2}$$

a multilinear evaluation — which the baseline nevertheless computed with the quadratic side's machinery: a bit transpose of every 8192-bit window plus 32 of the drain's 48 gathers per lane.

By (1), a *true* multilinear fold at the actual pinned challenges reproduces the weights in (2) exactly. The prover already materializes the *lincheck stripe*, a repacking of z indexed $[\, i_{\text{inner}} \,|\, i_{\text{outer}} \,]$, for the later lincheck phase. So:

1: $v \leftarrow \mathrm{fold\_stripe\_outer}(z_{\text{stripe}},\ \mathrm{eq}(r_{k_{\log}..m}))$ ▷ the one $O(2^m)$ pass; same kernel lincheck uses
2: **for** $d = k_{\log} - 1$ **downto** 7 **do** ▷ data halves each round: $2^{14} \to 2^7$ elements
3: $\quad v[i] \leftarrow v[i] + r_d\,(v[i] + v[i + 2^d])$
4: **end for**
5: $\mathrm{bank}_p[\lambda] \leftarrow \dfrac{\mathrm{eq}(r_6, p)}{C_s}\; v[\, p \cdot 64 + \lambda \,]$ ▷ $C_s = (1{+}r_6)(1{+}r_7)(1{+}r_8)$, from r itself

Dimension 6 (the parity) and dimensions 0..5 (the lanes) are kept unfolded; the constant in step 3 undoes the two eq factors the partial fold applied, derived from the identity $Y_p = (C_s/\mathrm{eq}(r_6, p))\,\mathrm{bank}_p$ obtained by factoring (1) over the folded dimensions. With the banks gone from the drain, the workers run AB-only: the per-window bit transpose is deleted and the drain issues one gather per (lane, b) instead of three. Equivalence was proven at two levels before wiring: banks bit-identical to the drain's, and all three entry outputs identical, dense and padded.

Measured: round 1 1277.5 → 1204.7 *ms* (−5.7%), 8/8. *The cost of the added fold* (≈ 180 *ms*) *is included; the removals* (≈ 250 *ms*) *net out.*

7 Register-resident round 2

Round 2's message loop crossed the vector/general register-file boundary three times per output pair: the table-driven fold extracted its vector accumulator into a two-word struct; the message products $g_1 = a_1 b_1$, $g_\infty = (a_0{+}a_1)(b_0{+}b_1)$ went through the scalar field multiply (struct in, struct out); and the accumulate of Section 2 re-entered the vector file. The rewrite keeps the whole chain in vector registers: the fold returns its register unextracted; the products use an in-register Karatsuba multiply (five `PMULL`s — three for the product, two for the fold through $X^{128} \equiv X^7{+}X^2{+}X{+}1$ — against the baseline's six); the $\mathrm{eq} \cdot g$ products feed the Section 2 accumulator directly. Per pair, the only remaining memory traffic is the four required stores of folded values and one eq load.

Measured: round-2 phase 358.1 → 311.6 *ms* (−13.0%), 8/8, *every pair in* [−12.6%, −13.3%], *taken on a machine with an active browser session (the min-of-5 protocol absorbs interactive bursts).*

The rounds-3+ tail: fusing away a memory pass. The tail rounds (3 and beyond) apply the same treatment plus one thing round 2 had no opportunity for. The incumbent made

two passes per worker chunk: fold $a_{\text{in}}, b_{\text{in}}$ into $a_{\text{out}}, b_{\text{out}}$, then re-read the just-written multi-megabyte chunk to build the message — an L2/DRAM read of data that had been in vector registers moments earlier. The fused kernel folds each output pair in-register ($e + r\,(e + o)$ via `mul_q`), issues the one required store, and consumes the pair for the message while still in registers:

1: **for** $x = 0 \ldots lo - 1$ **do**
2: $\quad a_0, a_1, b_0, b_1 \leftarrow$ fold four input pairs at r ▷ in q registers
3: $\quad$ store a_0, a_1, b_0, b_1 ▷ the required write, once
4: $\quad g_1 \leftarrow a_1 b_1$; $\quad g_\infty \leftarrow (a_0{+}a_1)(b_0{+}b_1)$ ▷ `mul_q`, never leaving registers
5: $\quad acc_1 \oplus{=} \text{eq}_x \odot g_1$; $\quad acc_\infty \oplus{=} \text{eq}_x \odot g_\infty$ ▷ unreduced, §2
6: **end for**

The `PMULL` count is identical to the two-pass form; the entire gain is one traversal instead of two plus the removed struct crossings. Notably, two earlier attempts on this exact loop had failed (a memory-interfaced pair-multiply kernel, +10%; wide accumulators alone, 0%): they changed the arithmetic encoding and the accumulator while leaving the pass structure intact, which is where the cost actually lived.

Measured: rounds-3+ phase 449.5 → 306.6 *ms (*−31.8%*),* 8/8*, every pair in* [−31.1%, −32.6%] *— the largest single win of the effort.*

8 Summary

Optimization	Kind	Phase effect (ST)	Si
§2 deferred reduction in registers	representation	round 2: −13.4%	d
§3 structurally-zero b rows	work removal	round 1: −2.9%	
§4 two-lane drain	ILP	round 1: −4.3%	
§5 unreduced accumulate, split weights	arithmetic	round 1: −9.1%	
§6 C side as stripe fold	structural	round 1: −5.7%	
§7 register-resident round 2	representation	round 2: −13.0%	
§7 fused rounds-3+ tail	representation + pass fusion	rounds 3+: −31.8%	
round 1 cumulative		1477 → 1205 ms	(−
round 2 cumulative (§2+§7)		433 → 312 ms	(
rounds 3+ cumulative		455 → 307 ms	(
end-to-end, all seven		≈ 52,400 →≈ 58,100 comp/s	(≈ +11%

Negative space. The kept changes are one side of a 17-experiment record; the failures shaped them. On this core, re-encoding fixed work never paid: pairing multiplies through a memory-interfaced kernel (+10%), a four-register structured load (+12%, microcoded), three-way XOR fusion (+1.8%), an 8× smaller gather table (−3.6% regression), and deferring an already-cheap reduction (0%) all failed, while every win either removed work, added independent work in flight, or removed register-file boundary crossings. Two further structural transplants measured neutral end-to-end and were reverted despite making the zerocheck *bucket* look better (hoisting the challenge-independent prep under the commit; the lookahead double-round, which loses 7–15% in the zerocheck tail on both this machine and the reference tree's, while *helping* the PCS

open by 7% — the same technique, opposite signs, two call sites).